

Array objects ^

[The N-dimensional array \(ndarray\)](#)

[Scalars](#)

[Data type objects \(dtype\)](#)

Indexing routines ^

[numpy.c](#)

[numpy.r](#)

[numpy.s](#)

[numpy.nonzero](#)

[numpy.where](#)

[numpy.indices](#)

[numpy.ix](#)

[numpy.ogrid](#)

[numpy.ravel multi index](#)

[numpy.unravel_index](#)

[numpy.diag_indices](#)

[numpy.diag_indices from](#)

[numpy.mask_indices](#)

[numpy.tril_indices](#)

[numpy.tril_indices from](#)

[numpy.triu_indices](#)

[numpy.triu_indices from](#)

[numpy.take](#)

[numpy.take_along_axis](#)

[numpy.choose](#)

[numpy.compress](#)

[numpy.diag](#)

[numpy.diagonal](#)

[numpy.select](#)

[numpy.lib.stride_tricks.sliding](#)

[numpy.lib.stride_tricks.as_strided](#)

[numpy.place](#)

[numpy.put](#)

[numpy.put_along_axis](#)

[numpy.putmask](#)

[numpy.fill_diagonal](#)

[numpy.nditer](#)

[numpy.ndenumerate](#)

[numpy.ndindex](#)

[numpy.nested_iters](#)

[numpy.flatiter](#)

[numpy.lib.Arrayiterator](#)

[numpy.iterable](#)

[Iterating Over Arrays](#)

numpy.r_

`numpy.r_ = <numpy.lib.index_tricks.RClass object>`

Translates slice objects to concatenation along the first axis.

This is a simple way to build up arrays quickly. There are two use cases.

1. If the index expression contains comma separated arrays, then stack them along their first axis.
2. If the index expression contains slice notation or scalars then create a 1-D array with a range indicated by the slice notation.

If slice notation is used, the syntax `start:stop:step` is equivalent to `np.arange(start, stop, step)` inside of the brackets. However, if `step` is an imaginary number (i.e. 100j) then its integer portion is interpreted as a number-of-points desired and the start and stop are inclusive. In other words `start:stop:stepj` is interpreted as `np.linspace(start, stop, step, endpoint=1)` inside of the brackets. After expansion of slice notation, all comma separated sequences are concatenated together.

Optional character strings placed as the first element of the index expression can be used to change the output. The strings 'r' or 'c' result in matrix output. If the result is 1-D and 'r' is specified a 1 x N (row) matrix is produced. If the result is 1-D and 'c' is specified, then a N x 1 (column) matrix is produced. If the result is 2-D then both provide the same matrix result.

A string integer specifies which axis to stack multiple comma separated arrays along. A string of two comma-separated integers allows indication of the minimum number of dimensions to force each entry into as the second integer (the axis to concatenate along is still the first integer).

A string with three comma-separated integers allows specification of the axis to concatenate along, the minimum number of dimensions to force the entries to, and which axis should contain the start of the arrays which are less than the specified number of dimensions. In other words the third integer allows you to specify where the 1's should be placed in the shape of the arrays that have their shapes upgraded. By default, they are placed in the front of the shape tuple. The third argument allows you to specify where the start of the array should be instead. Thus, a third argument of '0' would place the 1's at the end of the array shape. Negative integers specify where in the new shape tuple the last dimension of upgraded arrays should be placed, so the default is '-1'.

Parameters: Not a function, so takes no parameters

Returns: A concatenated ndarray or matrix.

i See also

[concatenate](#)

Join a sequence of arrays along an existing axis.

[c_](#)

Translates slice objects to concatenation along the second axis.

Examples

```
>>> np.r_[np.array([1,2,3]), 0, 0, np.array([4,5,6])]
array([1, 2, 3, ..., 4, 5, 6])
>>> np.r_-1:1:6j, [0]*3, 5, 6]
array([-1. , -0.6, -0.2,  0.2,  0.6,  1. ,  0. ,  0. ,  0. ,  5. ,  6. ])
```

String integers specify the axis to concatenate along or the minimum number of dimensions to force entries into.

```
>>> a = np.array([[0, 1, 2], [3, 4, 5]])
>>> np.r_['-1', a, a] # concatenate along last axis
array([[0, 1, 2, 0, 1, 2],
       [3, 4, 5, 3, 4, 5]])
>>> np.r_['0,2', [1,2,3], [4,5,6]] # concatenate along first axis, dim>=2
array([[1, 2, 3],
       [4, 5, 6]])
```

```
>>> np.r_['0,2,0', [1,2,3], [4,5,6]]
array([[1],
       [2],
       [3],
       [4],
       [5],
       [6]])
>>> np.r_['1,2,0', [1,2,3], [4,5,6]]
array([[1, 4],
       [2, 5],
       [3, 6]])
```

Using 'r' or 'c' as a first string argument creates a matrix.

```
>>> np.r_['r',[1,2,3], [4,5,6]]
matrix([[1, 2, 3, 4, 5, 6]])
```

< Previous
[numpy.c_](#)

Next >
[numpy.s_](#)